

Pilot Driving School Project Documentation

1) Project Overview

Pilot is a driving school and training platform.

It helps users explore Car, Bike, and License packages, then submit enquiry details or proceed to payment for selected services.

The goal is to make learner onboarding and booking smoother and faster.

2) Problem Statement

Many driving schools still manage learner enquiries, package details, and bookings manually through calls and chats.

This causes delays, missing information, and low conversion.

Pilot solves this by giving a clear digital flow for package discovery, enquiry submission, and payment support.

3) Use Cases

- New users compare training packages before joining.
- Learners submit details for free trial or instructor callback.
- Users choose zone/location and preferred service path.
- License-focused users can move directly to payment.
- Business team receives structured leads for faster follow-up.

4) Features List

- Responsive landing page with service information
- Package listing with tabs (Car, Bike, License)
- Package filtering and sorting
- Recommended package highlighting
- Package detail page with add-on selection
- Price summary with tax and total
- Lead enquiry modal with form validation
- Dynamic zone loading from API
- Login and registration UI with validation
- License package payment redirection
- Legal and policy pages
- Account deletion request flow

5) Technical Implementation (Step by Step)

1. User opens the app and views landing content.
2. User goes to Packages page.
3. Frontend requests packages from internal API route.
4. API route fetches data from upstream service with timeout handling.
5. Frontend filters active packages and shows category-wise options.
6. User opens package detail page and reviews selected package.
7. User selects optional add-ons (for non-license flow).
8. Lead form opens and fetches zones dynamically.
9. Form validates inputs (name, phone, zone).
10. Form submits enquiry payload to backend route.
11. Backend route forwards data to upstream enquiry service.
12. For license flow, user is redirected to payment link.

6) Tech Stack Used

- Next.js (App Router)
- React
- Tailwind CSS
- Formik
- Yup
- Lucide React (icons)
- Next.js API Route Handlers (Node runtime)
- External REST APIs

7) Challenges Faced and Solutions

Challenge 1: API timeout and unstable responses

Solution: Added AbortController timeout, safe parsing, and fallback error handling.

Challenge 2: Inconsistent API response shapes

Solution: Created normalization functions for packages, add-ons, and zones.

Challenge 3: UX on mobile and desktop

Solution: Built responsive layouts, loading states, empty states, and retry handling.

Challenge 4: Form quality and bad user input

Solution: Added Yup validation and controlled submission flow.

8) Improvements / Future Scope

- Full auth backend with secure login/session flow
- Real-time slot booking calendar
- Instructor dashboard and learner dashboard
- Simulation progress and performance tracking
- Payment status verification with webhook integration
- Admin analytics dashboard
- Automated unit/integration/e2e tests

9) 1-Minute Interview Explanation

I built a project called Pilot, which is a driving school and training platform. The system helps users browse car, bike, and license training packages, compare options, and quickly take action.

Users can open package details, choose add-ons, and submit enquiry forms with name, phone, and zone. For license packages, users can directly continue to payment.

I used Next.js and React for frontend, Tailwind for UI, and Formik plus Yup for form validation. I also built internal API routes to connect with external services and handle errors consistently.

A key challenge was unstable API behavior, so I added timeout handling and defensive parsing to keep the app reliable. Overall, this project solves real business and user pain by digitizing the learner booking journey.

10) 2-3 Minute Detailed Explanation

Pilot is a driving training platform designed to improve how driving schools manage leads and user onboarding. In many local setups, enquiries and bookings are still manual, which is slow and unstructured.

In this project, the user starts at the landing page and then explores training packages by category: Car, Bike, and License. Package data is fetched from API and displayed in a clean, filterable UI.

When the user opens package details, they can view pricing, package information, and add-ons. For car and bike packages, they can submit a lead form. The form includes name, phone, and zone, with validation and proper error handling.

Zones are loaded from API dynamically, and submission goes through a backend route that forwards data to upstream service. For license packages, the action goes directly to payment link flow.

Technically, I used Next.js App Router, React, Tailwind CSS, Formik, and Yup. I implemented route handlers for packages, zones, enquiry, and account-deletion related requests.

The main engineering challenge was handling unreliable API responses and latency. I solved this with timeout control, safe parsing, normalized response shapes, and stable frontend states.

In future, I plan to add full authentication, slot booking calendar, learner progress tracking, admin analytics, and automated tests. This project demonstrates frontend development, API integration, product flow design, and practical problem-solving.

11) Video Script (Ready to Speak)

Hi everyone, today I am presenting my project, Pilot.

Pilot is a driving school platform where users can explore training packages, submit enquiry details, and for selected services continue to payment.

The problem I focused on is that many driving schools still use manual call-based processes. That creates delays and unstructured lead management.

In Pilot, users can browse packages in three categories: Car, Bike, and License. This makes package discovery simple and user-friendly.

When a user opens a package, they can view details and choose add-ons. For car and bike plans, the user can submit an enquiry form. The form takes full name, phone number, and zone, and includes validation.

Zones are loaded from API, and the enquiry is sent through backend route handlers to upstream services. For license packages, users can directly proceed to payment flow.

I built this using Next.js, React, Tailwind CSS, Formik, and Yup. I also implemented API routes with timeout and error-handling logic for better reliability.

One key challenge was unstable API behavior. I handled that with request timeouts, defensive response parsing, and consistent error handling.

In future, I want to add full authentication, slot booking calendar, learner progress tracking, and admin analytics.

Thank you for watching. This is my Pilot project.